

# CodeAlpha C++ Programming Internship

Task Submissions | Siraj Ahmed

## Task 1: CGPA Calculator

This program takes the number of courses a student has taken, then for each course reads the grade points and credit hours. It multiplies grade points by credit hours to get the quality points for each course, sums them up, and divides by the total credit hours to compute the final CGPA. It also prints a summary of each course before showing the result.

```
#include <iostream>
using namespace std;

int main() {
    int numCourses;
    cout << "Enter number of courses: ";
    cin >> numCourses;

    float grades[numCourses];
    int creditHours[numCourses];
    float totalGradePoints = 0;
    int totalCredits = 0;

    for (int i = 0; i < numCourses; i++) {
        cout << "\nCourse " << i + 1 << ":\n";
        cout << "Enter grade points (e.g. 4.0, 3.7, 3.3): ";
        cin >> grades[i];
        cout << "Enter credit hours: ";
        cin >> creditHours[i];

        totalGradePoints += grades[i] * creditHours[i];
        totalCredits += creditHours[i];
    }

    float cgpa = totalGradePoints / totalCredits;

    cout << "\n----- Result ----- \n";
    for (int i = 0; i < numCourses; i++) {
        cout << "Course " << i + 1 << ": Grade = " << grades[i]
            << ", Credit Hours = " << creditHours[i] << endl;
    }
    cout << "Total Credit Hours: " << totalCredits << endl;
    cout << "Final CGPA: " << cgpa << endl;

    return 0;
}
```

## Task 2: Login and Registration System

This program stores usernames and passwords in a text file. During registration, it checks whether the username already exists before saving new credentials. During login, it reads through the file and matches the entered username and password against the stored records, then shows a success or error message accordingly.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```

bool userExists(string username) {
    ifstream file("users.txt");
    string storedUser, storedPass;
    while (file >> storedUser >> storedPass) {
        if (storedUser == username) {
            file.close();
            return true;
        }
    }
    file.close();
    return false;
}

void registerUser() {
    string username, password;
    cout << "Enter username: ";
    cin >> username;

    if (userExists(username)) {
        cout << "Username already exists! Try a different one.\n";
        return;
    }

    cout << "Enter password: ";
    cin >> password;

    ofstream file("users.txt", ios::app);
    file << username << " " << password << endl;
    file.close();

    cout << "Registration successful!\n";
}

void loginUser() {
    string username, password;
    cout << "Enter username: ";
    cin >> username;
    cout << "Enter password: ";
    cin >> password;

    ifstream file("users.txt");
    string storedUser, storedPass;
    bool found = false;

    while (file >> storedUser >> storedPass) {
        if (storedUser == username && storedPass == password) {
            found = true;
            break;
        }
    }
    file.close();

    if (found)
        cout << "Login successful! Welcome, " << username << ".\n";
    else
        cout << "Invalid username or password.\n";
}

int main() {
    int choice;
    do {
        cout << "\n1. Register\n2. Login\n3. Exit\nEnter choice: ";
        cin >> choice;
    }
}

```

```

        if (choice == 1) registerUser();
        else if (choice == 2) loginUser();
        else if (choice == 3) cout << "Goodbye!\n";
        else cout << "Invalid choice.\n";

    } while (choice != 3);

    return 0;
}

```

### Task 3: Banking System

This program uses three classes: Transaction, Account, and Customer. Each account keeps a running balance and a list of past transactions. Customers can deposit, withdraw, and transfer money to other customers, and every action is logged so the transaction history can be displayed later.

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Transaction {
public:
    string type;
    double amount;

    Transaction(string t, double a) {
        type = t;
        amount = a;
    }
};

class Account {
public:
    int accountNumber;
    double balance;
    vector<Transaction> history;

    Account(int accNum, double initialBalance) {
        accountNumber = accNum;
        balance = initialBalance;
    }

    void deposit(double amount) {
        balance += amount;
        history.push_back(Transaction("Deposit", amount));
        cout << "Deposited " << amount << ". New balance: " << balance << endl;
    }

    void withdraw(double amount) {
        if (amount > balance) {
            cout << "Insufficient balance!\n";
            return;
        }
        balance -= amount;
        history.push_back(Transaction("Withdraw", amount));
        cout << "Withdrew " << amount << ". New balance: " << balance << endl;
    }

    void showHistory() {
        cout << "\nTransaction History for Account " << accountNumber << ":\n";
    }
};

```

```

        for (auto &t : history) {
            cout << t.type << ": " << t.amount << endl;
        }
    }
};

class Customer {
public:
    string name;
    Account account;

    Customer(string n, int accNum, double initialBalance)
        : name(n), account(accNum, initialBalance) {}

    void transferTo(Customer &receiver, double amount) {
        if (amount > account.balance) {
            cout << "Insufficient balance for transfer!\n";
            return;
        }
        account.balance -= amount;
        receiver.account.balance += amount;

        account.history.push_back(Transaction("Transfer Out", amount));
        receiver.account.history.push_back(Transaction("Transfer In", amount));

        cout << "Transferred " << amount << " to " << receiver.name << endl;
    }
};

int main() {
    Customer c1("Ali", 101, 5000);
    Customer c2("Sara", 102, 3000);

    int choice;
    do {
        cout << "\n--- Banking System ---\n";
        cout << "1. Deposit (Ali)\n2. Withdraw (Ali)\n3. Transfer Ali->Sara\n";
        cout << "4. Show Ali's History\n5. Show Sara's History\n6. Show Balances\n7. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        double amt;
        switch (choice) {
            case 1:
                cout << "Enter amount: ";
                cin >> amt;
                c1.account.deposit(amt);
                break;
            case 2:
                cout << "Enter amount: ";
                cin >> amt;
                c1.account.withdraw(amt);
                break;
            case 3:
                cout << "Enter amount: ";
                cin >> amt;
                c1.transferTo(c2, amt);
                break;
            case 4:
                c1.account.showHistory();
                break;
            case 5:
                c2.account.showHistory();

```

```
        break;
    case 6:
        cout << "Ali's Balance: " << c1.account.balance << endl;
        cout << "Sara's Balance: " << c2.account.balance << endl;
        break;
    case 7:
        cout << "Exiting...\n";
        break;
    default:
        cout << "Invalid choice.\n";
    }
} while (choice != 7);

return 0;
}
```

*All three programs were written and tested individually in a standard C++ compiler (g++). Comments were kept minimal on purpose, to keep the logic easy to trace through while going over the code.*